FlameIQ

PREDICTION MODEL SPECIFICATION

MVP Data Science Design for Hardware-Free LPG Refill Prediction

Prepared by	Chimdinma Ihetu | Data Science
Product	FlameIQ
Status	Hackathon MVP Specification

Source basis: FlameIQ Market Analysis and survey findings (113 responses, Nigeria, August 2026).

Document Control

Item	Detail
Document purpose	Define how FlameIQ will predict a household's next LPG refill need without physical sensors.
Primary audience	Product management, backend engineering, AI/ML, virtual assistant, data science, and technical judges.
MVP model owner	Data Science Team
Approval required from	Product Management and Backend Engineering
Data maturity	Survey validation available; longitudinal refill-cycle training data not yet available.
Prototype constraint	Synthetic data may be used only for proof-of-concept development and must be labelled clearly.

Purpose of This Specification

This document converts FlameIQ's central product promise into an implementable data science plan. It defines the prediction target, the minimum data to collect, the cold-start method, baseline logic, candidate models, evaluation criteria, system integration requirements, and the handoff boundaries between Data Science and the other hackathon functions.

> **Core product promise**
> FlameIQ should tell a user when their cooking gas is likely to finish, without requiring a physical gas-level sensor, and convert that prediction into a timely refill reminder or order.

Contents

1. Business Context and Scope

FlameIQ is positioned as an AI-powered LPG marketplace for Nigerian households. The product strategy identifies unexpected gas depletion as a frequent user problem and hardware-free prediction as the main acquisition wedge. The same strategy also makes clear that prediction will not retain users by itself: transparent pricing, verified vendors, and reliable fulfilment remain operational requirements.

The survey validates interest in prediction, but it does not provide the repeated, timestamped refill cycles required to train a production model. Therefore, this specification separates three different evidence layers:

- Market validation data: survey responses showing the problem exists and users value prediction.
- Prototype modelling data: synthetic refill-cycle records used to demonstrate the technical concept during the hackathon.
- Production learning data: real user refill and order history collected after launch and used to personalise the model.

> **Critical distinction**
> A model trained on synthetic data can demonstrate the workflow, API, user experience, and technical feasibility. It cannot prove real-world accuracy. Production accuracy must be established through a pilot using real refill cycles.

1.1 In Scope for the MVP

- Predict the expected number of days until a household needs another LPG refill.
- Return an estimated refill date and a confidence level.
- Generate a short, understandable explanation for the prediction.
- Support a chatbot or app reminder that can lead directly to ordering.
- Capture user feedback and completed refill events so the model improves over time.

1.2 Out of Scope for the MVP

- Real-time physical measurement of LPG quantity inside a cylinder.
- Computer vision inspection of cylinders, regulators, or hoses.
- National supplier demand forecasting.
- Dynamic LPG pricing.
- Commercial restaurant and multi-cylinder optimisation, because the current research under-represents business users.
- Any claim that the prediction is a safety device or a certified measurement of remaining LPG.

2. Prediction Problem Definition

2.1 Primary Prediction Target

The MVP should be framed as a supervised regression problem.

> **Primary target**
> days_until_next_refill = the number of calendar days from the prediction date until the user completes the next refill.

The system will convert the numeric prediction into a user-facing estimated refill date. For example, if the model predicts 6 days remaining on 10 August, the estimated refill date is 16 August.

2.2 Secondary Decision Output

A secondary rule can convert the prediction into a binary action: refill needed within the next seven days - yes or no. This should not be trained as a separate model during the MVP unless time permits. It can be derived from the regression output.

2.3 Unit of Observation

Each modelling row should represent one completed refill cycle for one user and one cylinder profile. A refill cycle starts when the cylinder is filled and ends when the user completes the next refill.

user_id	cycle_start	cycle_end	cylinder_kg	household_size	meals_per_day	target_days
U001	01-Jul-2026	28-Jul-2026	12.5	4	2	27
U002	05-Jul-2026	20-Jul-2026	6	5	3	15
U003	10-Jul-2026	07-Aug-2026	12.5	2	1	28

2.4 Modelling Assumptions

- A completed refill is used as the practical proxy for the previous cylinder becoming empty or nearly empty.
- Users may refill early, partially refill, use a second cylinder, or switch cooking fuels. These behaviours must be captured where possible because they distort the target.
- The prediction is an estimate of likely refill need, not a direct measurement of gas remaining.
- For users with irregular consumption, the system must show wider uncertainty and avoid false precision.

3. User Scenarios and Model Behaviour

Scenario	Available data	Prediction method	Expected confidence
New user	Onboarding answers and last refill date only	Cohort/rule-based estimate	Low
User with 1 completed cycle	Profile plus one observed interval	Blend of cohort estimate and observed interval	Low to moderate
User with 2-3 completed cycles	Profile plus personal history	Personalised baseline or ML model	Moderate
Established user	Multiple recent refill cycles	Personalised model with recency weighting	Moderate to high
Irregular user	High variation, partial fills, events, or fuel switching	Conservative estimate with wider range	Low

3.1 Expected User Experience

The prediction should not appear as an unexplained number. The product should present a range, a confidence label, and a short reason. The user should be able to correct wrong assumptions, confirm a refill, or dismiss a reminder. Those actions become new training signals.

4. Data Requirements

The minimum viable dataset should be small enough to avoid onboarding friction but complete enough to support a meaningful estimate. Fields are classified as mandatory, recommended, optional, or derived.

4.1 User and Household Inputs

Field	Type	Source	Priority	Predictive purpose
user_id	String/UUID	Backend	Mandatory	Links profile, orders, and refill cycles.
household_size	Integer	Onboarding/VA	Recommended	Proxy for cooking demand.
meals_per_day	Integer 1-5	Onboarding/VA	Recommended	Captures cooking intensity.
cooking_days_per_week	Integer 1-7	Onboarding/VA	Optional	Separates daily from occasional use.
primary_cooking_fuel	Category	Onboarding/VA	Recommended	Identifies mixed-fuel households.
commercial_use	Boolean	Onboarding/VA	Mandatory	Excludes or flags business usage for the household MVP.
typical_refill_frequency	Category/days	Onboarding/VA	Recommended for new users	Provides a cold-start anchor.
location_zone	Coarse category	Profile/order	Optional	Supports cohort grouping and operations without requiring precise home coordinates.

4.2 Cylinder and Refill Inputs

Field	Type	Source	Priority	Predictive purpose
cylinder_size_kg	Numeric/category	Onboarding/order	Mandatory	Main capacity indicator.
number_of_cylinders	Integer	Onboarding	Recommended	Detects spare-cylinder use.
refill_date	Date-time	Order/manual log	Mandatory	Defines the start of a refill cycle.
quantity_refilled_kg	Numeric	Order/vendor	Mandatory	Separates full from partial refill.
fill_type	Full/partial	Order/user confirmation	Recommended	Prevents partial fills from appearing as full cycles.
order_channel	App/manual/other	Backend	Optional	Identifies refills completed outside FlameIQ.
second_fuel_used	Boolean/category	Check-in	Optional	Explains unusually long LPG cycles.
high_usage_event	Boolean/category	Check-in	Optional	Explains unusually short cycles caused by visitors, parties, baking, or bulk cooking.

4.3 Derived Features

Derived feature	Definition	Reason
days_since_last_refill	Prediction date minus latest refill date	Current cycle progress.
previous_cycle_days	Days between the two most recent refills	Strong personal baseline.
mean_cycle_days_3	Mean of the last three completed cycles	Smooths one-off variation.
median_cycle_days	Median of all valid cycles	Robust to unusual events.
cycle_variability	Standard deviation or median absolute deviation	Used to estimate confidence.
refills_last_90_days	Count of valid refills in the prior 90 days	Captures recent intensity.
kg_per_person	Cylinder or refill quantity divided by household size	Normalises capacity by household demand.
estimated_daily_usage	Refill quantity divided by prior cycle duration	Interpretable consumption proxy.
month/season flags	Calendar features from cycle start	Tests seasonal or festive cooking effects.

4.4 Fields Not Required for the MVP

The model does not need the user's full income, religion, ethnicity, exact home address, contact list, or unrelated demographic data. Collecting data without a defined predictive or operational purpose creates privacy risk and onboarding friction.

5. Data Collection and Product Instrumentation

5.1 Onboarding Flow

The first estimate can be generated after a short onboarding sequence. The VA or app should collect only the minimum information required: cylinder size, last refill date, household size, meals per day, primary cooking fuel, and typical refill frequency. Users should be allowed to skip non-essential questions, but the product must show that skipped inputs may reduce accuracy.

5.2 Transaction and Refill Events

Event name	Trigger	Required fields	Model use
profile_completed	User completes onboarding	user_id, timestamp, household and cylinder fields	Cold-start features.
refill_logged	User records or orders a refill	user_id, refill_date, cylinder_size, quantity, fill_type	Starts a new cycle.

Event name	Trigger	Required fields	Model use
refill_confirmed	Delivery completed or manual refill confirmed	order_id, actual date, quantity	High-quality cycle start.
prediction_generated	System creates an estimate	model_version, predicted_days, range, confidence	Audit and performance tracking.
reminder_sent	Prediction crosses reminder threshold	prediction_id, channel, timestamp	Measures intervention timing.
reminder_action	User orders, dismisses, or snoozes	action, timestamp	Behaviour and trust signal.
gas_finished_reported	User reports actual empty date	date, optional context	Best available outcome label.
prediction_feedback	User rates estimate	too_early/accurate/too_late	Calibration and product usefulness.

5.3 Data Quality Rules

- Reject impossible dates, such as a cycle end before its start.
- Flag refill quantities greater than the registered cylinder capacity for review.
- Do not treat partial refills as full-cylinder cycles without adjustment.
- Do not combine two cylinders into one cycle unless the product explicitly supports multi-cylinder profiles.
- Retain an audit trail when users edit refill dates or cylinder details.
- Mark cycles as incomplete until a next refill or gas-finished event is observed.

6. Training Dataset Design

6.1 Target Construction

For a valid completed cycle:

> **Target formula**
> target_days = next_valid_refill_date - current_refill_date

A valid cycle should normally represent a known refill quantity and a stable cylinder profile. If the user changes cylinder size, adds another cylinder, or reports a partial refill, the cycle should be split, adjusted, or flagged rather than treated as directly comparable.

6.2 Incomplete and Censored Cycles

A current cycle has no known next refill date and therefore cannot be used as a supervised training label. It remains useful for generating a prediction. For production analytics, such cycles may later be handled with survival analysis, but survival modelling is not required for the hackathon MVP.

6.3 Synthetic Prototype Dataset

Because the available survey is not longitudinal, the hackathon prototype may use a synthetic dataset representing plausible household refill cycles. The synthetic generator should preserve logical relationships rather than produce random columns with no physical meaning.

- Larger cylinders should generally last longer, all else equal.
- Larger households and more meals per day should generally shorten the refill interval.
- Mixed-fuel use should generally lengthen the LPG refill interval.
- High-usage events and commercial use should shorten the interval.
- Random variation should be added to avoid a perfectly deterministic dataset.
- Missing values, partial fills, and a small number of outliers should be included to test robustness.

> **Disclosure requirement**
> Any accuracy reported from synthetic data must be labelled "prototype performance on synthetic data". It must not be presented as evidence of real household prediction accuracy.

7. Cold-Start Strategy

The cold-start problem occurs because a new user has no FlameIQ refill history. The MVP should not pretend that the model is fully personalised on day one. It should use a staged approach.

Stage	Available evidence	Method	Message to user
Stage 0: New user	Profile and self-reported typical frequency	Cohort median plus rule adjustment	Initial estimate - accuracy improves after each refill.
Stage 1: One cycle	One observed personal interval	Weighted blend of personal interval and cohort estimate	Learning your pattern.
Stage 2: Two to three cycles	Early refill history	Personalised baseline or ML prediction	Personalised estimate with moderate confidence.
Stage 3: Established user	Multiple recent cycles	Personalised model with recency and variability features	Higher-confidence estimate when usage is stable.

7.1 Initial Rule-Based Estimate

The initial estimate should begin with the user's stated typical refill interval when available. If the user does not know it, use the median interval of similar synthetic or pilot users based on cylinder size, household size, meals per day, and primary cooking fuel. Apply conservative adjustments rather than aggressive precision.

7.2 Confidence Logic

Confidence	Typical condition	User presentation
Low	No personal history, conflicting answers, or highly variable usage	Show a wider date range and state that the estimate is still learning.
Moderate	Two or more coherent cycles with moderate variation	Show a narrower range and the main reasons.
High	Several recent, consistent cycles and no major profile change	Show a narrow range, while still labelling it as an estimate.

8. Baselines and Candidate Models

8.1 Required Baselines

Baseline	Definition	Purpose
Naive last-cycle baseline	Predict the next cycle will equal the most recent valid cycle.	Simple personalised reference.
Personal median baseline	Predict the median of the user's valid prior cycles.	Robust personalised reference.
Cohort median baseline	Predict the median interval for similar users.	Cold-start reference.
Weighted hybrid baseline	Blend personal and cohort estimates based on number of observed cycles.	Strong practical MVP baseline.

A machine-learning model should not be deployed merely because it is more sophisticated. It must outperform the strongest relevant baseline on held-out users and should produce predictions that are stable enough to explain.

8.2 Candidate Models

Model	Use in MVP	Strength	Risk/limitation
Linear Regression / Elastic Net	Recommended benchmark	Transparent and easy to explain.	May miss non-linear relationships.
Random Forest Regressor	Recommended candidate	Handles non-linear interactions and mixed features.	Can overfit small synthetic datasets; less smooth.

Model	Use in MVP	Strength	Risk/limitation
Gradient Boosting Regressor	Recommended candidate	Strong tabular performance and feature interactions.	Requires careful tuning and explanation.
XGBoost/LightGBM	Optional if available	High performance on structured data.	Unnecessary complexity for a small hackathon dataset.
Neural Network	Not recommended	Flexible model family.	No justification for current data volume or problem complexity.

Recommended MVP path

Build the weighted hybrid baseline first. Then compare Linear Regression, Random Forest, and Gradient Boosting. Select the simplest model that produces the best reliable performance on held-out users.

9. Modelling and Validation Workflow

1. Create the modelling table at one row per completed refill cycle.
2. Validate dates, capacity, refill quantity, and cycle completeness.
3. Separate features available at prediction time from outcome information.
4. Split the data by user, not randomly by row, to prevent the same household appearing in both training and testing.
5. Where enough history exists, preserve time order so future cycles are not used to predict past cycles.
6. Train and evaluate the baseline models.
7. Train the candidate regression models using the same split.
8. Compare absolute error, late-prediction rate, stability, and explainability.
9. Package the selected model with preprocessing and version metadata.
10. Expose the model through a backend API and log every prediction.

9.1 Leakage Controls

- Never use the next refill date as an input feature; it is the outcome.
- Do not use future orders, future ratings, or future delivery information to predict an earlier cycle.
- Do not use post-refill feedback generated after the target date as a model input.
- When generating synthetic data, avoid deriving the target from a formula and then feeding the exact same formula components without noise; this produces misleadingly perfect results.

10. User-Facing Prediction and Explainability

10.1 Output Structure

Output field	Example	Purpose
estimated_days_remaining	6	Primary numeric prediction.
estimated_refill_date	16-Aug-2026	Human-readable action date.
prediction_range	14-19 Aug	Communicates uncertainty.
confidence	Moderate	Prevents false precision.
top_reasons	12.5 kg cylinder; 4-person household; recent cycles averaged 25 days	Builds trust.
recommended_action	Schedule refill within 4 days	Converts prediction into product value.

10.2 Chatbot Example

VA message

Your gas is estimated to need a refill in about 6 days, most likely between 14 and 19 August. This estimate is based on your 12.5 kg cylinder, household size, and recent refill pattern. Would you like to schedule a refill or update your

> usage details?

10.3 User Correction Loop

Every prediction should give the user a low-friction correction route. Examples include: "I refilled already", "I am using a second cylinder", "We cooked more than usual", "We used another fuel", or "This estimate is too early/too late". These corrections improve both data quality and user trust.

11. Evaluation Metrics and Acceptance Gates

11.1 Technical Metrics

Metric	Definition	Why it matters
Mean Absolute Error (MAE)	Average absolute difference between predicted and actual refill day.	Primary measure in days; easy to interpret.
Median Absolute Error	Median absolute prediction error.	More robust to extreme household behaviour.
Within +/- 3 days	Share of predictions no more than three days early or late.	Communicates practical usefulness.
Late-prediction rate	Share of predictions that occur after the actual refill need.	Late reminders create the core failure the product claims to prevent.
Early-prediction rate	Share of predictions materially earlier than actual need.	Too many early alerts create notification fatigue and distrust.
Baseline improvement	Percentage improvement over the selected naive/hybrid baseline.	Justifies the ML component.

11.2 Product Metrics

Metric	Interpretation
Prediction usefulness rating	Do users consider the estimate useful after actual refill?
Reminder action rate	Share of reminders that lead to order, schedule, or intentional snooze.
Prediction-to-order conversion	Share of model-triggered users who order through FlameIQ.
Correction rate	How often users report wrong assumptions or inaccurate estimates.
Repeat use after prediction	Whether prediction supports habitual reordering rather than one-time use.
Model coverage	Share of active users for whom the system can produce a valid estimate.

11.3 MVP Acceptance Gates

- The model must outperform the relevant cohort or hybrid baseline on held-out users.
- The system must return an uncertainty range and confidence label, not only a single date.
- The API must log model version, inputs, output, and timestamp for every prediction.
- The chatbot/app must allow user correction and refill confirmation.
- Synthetic-data results must be labelled clearly and not represented as real-world validation.
- Real deployment thresholds should be set after pilot data exists; they should not be invented from survey responses.

12. System Integration and API Contract

12.1 End-to-End Flow

User / VA	Backend	Feature Builder	Prediction Model	Response API	VA / App	Order / Feedback

12.2 Proposed Prediction Endpoint

Endpoint: POST /v1/predictions/refill

Example request


```
{
  "user_id": "U001",
  "prediction_date": "2026-08-10",
  "cylinder_size_kg": 12.5,
  "household_size": 4,
  "meals_per_day": 2,
  "primary_cooking_fuel": "LPG",
  "last_refill_date": "2026-07-21",
  "quantity_refilled_kg": 12.5,
  "recent_cycle_days": [24, 26, 25]
}
```


Example response


```
{
  "prediction_id": "PRED-82731",
  "model_version": "refill-v0.1",
  "estimated_days_remaining": 6,
  "estimated_refill_date": "2026-08-16",
  "prediction_window": {"start": "2026-08-14", "end": "2026-08-19"},
  "confidence": "moderate",
  "top_reasons": [
    "Recent refill cycles averaged 25 days",
    "12.5 kg cylinder",
    "Four-person household"
  ],
  "recommended_action": "Schedule a refill within four days"
}
```


12.3 Integration Requirements

- Backend must provide a stable user and cylinder identifier.
- Prediction service must receive only data available at prediction time.
- The service must return a safe fallback when required data is missing.
- Every response must include the model version and confidence.
- The VA should not invent a prediction when the API fails; it should provide a neutral fallback and invite the user to update their refill details.

13. Privacy, Fairness, and Model Risk

13.1 Privacy Requirements

- Collect only fields with a defined product or modelling purpose.
- Obtain clear consent for using refill and household data to personalise predictions.
- Use coarse location zones for modelling where exact coordinates are unnecessary.
- Separate personally identifiable information from the modelling table where possible.
- Define retention and deletion rules for user histories and prediction logs.

13.2 Fairness and Coverage

The model may perform differently across household sizes, cylinder capacities, locations, mixed-fuel users, and irregular cooks. Performance should be reported by relevant user segment once sufficient real data exists. A high overall score can hide poor performance for smaller subgroups.

13.3 Key Model Risks

Risk	Consequence	Mitigation
False precision	Users treat an estimate as a measurement.	Always show range, confidence, and estimate language.
Late reminder	User still runs out unexpectedly.	Track late-prediction rate and bias reminders conservatively.
Alert fatigue	Users ignore future reminders.	Limit early alerts; allow snooze and preference settings.

Risk	Consequence	Mitigation
Behaviour change	Guests, baking, second fuel, or cylinder change makes history unreliable.	Capture correction events and widen uncertainty.
Synthetic overfitting	Prototype appears unrealistically accurate.	Use realistic noise and disclose synthetic performance.
Data leakage	Test scores become meaningless.	Split by user/time and use only prediction-time features.

14. Monitoring and Continuous Improvement

14.1 Prediction Logging

For every prediction, retain the model version, feature timestamp, prediction, confidence, range, reminder decision, and eventual outcome when known. Without this audit trail, the team cannot measure whether accuracy improves after each refill.

14.2 Monitoring Dashboard Requirements

The Data Analytics team may visualise product and model monitoring, while Data Science defines the metrics and investigates model behaviour. Required monitoring views include MAE over time, late-prediction rate, error by cylinder size, error by number of personal cycles, confidence calibration, model coverage, and correction frequency.

14.3 Retraining Triggers

- A sufficient number of new valid refill cycles has accumulated.
- Prediction error worsens materially over time or within a key segment.
- The product changes the onboarding questions or refill workflow.
- New cylinder types, user segments, or launch locations are introduced.
- The production data distribution diverges from the original training data.

15. Hackathon Build Plan

Work package	Data Science output	Dependencies	Definition of done
1. Problem specification	Target, unit of observation, assumptions, scope	PM	Written specification approved.
2. Data schema	Feature dictionary and event requirements	PM, Backend, VA	Required fields mapped to source.
3. Synthetic data	Realistic refill-cycle dataset and generation assumptions	Survey/PM context	Dataset passes logical checks.
4. Baseline model	Cohort and weighted hybrid prediction	Synthetic dataset	Produces prediction, range, confidence.
5. Candidate models	Linear, Random Forest, Gradient Boosting comparison	Baseline complete	Selected model beats baseline or baseline retained.
6. Explainability	Top reasons and confidence logic	Selected model	User-facing message generated.
7. API package	Model pipeline or callable prediction function	Backend	Backend can send inputs and receive JSON output.
8. Demo evidence	Metric summary and limitations note	All teams	Pitch shows working flow without overstating accuracy.

15.1 Minimum Demonstrable MVP

- A user completes a short onboarding flow.
- The backend sends the user profile and refill history to the prediction service.
- The model returns estimated days remaining, refill date, range, confidence, and reasons.
- The chatbot presents the prediction and offers schedule, order, snooze, or correction actions.
- The system logs the prediction and the user action.

16. Team Boundaries and Handoffs

The Data Science role must remain distinct from Data Analytics and AI/ML while still collaborating with both.

Team	Primary responsibility	Data Science handoff/collaboration
Product Management	Problem, scope, personas, requirements, prioritisation	Approve target, onboarding burden, reminder policy, and product claims.
Data Analytics	Survey analysis, descriptive statistics, dashboards, business insights	Visualise model monitoring metrics defined by Data Science; provide data quality insights.
Data Science	Prediction target, feature design, dataset, baseline, model, validation, explainability	Own the refill prediction logic and performance evidence.
AI/ML	Broader AI feature proposal and technology recommendation	Align on deployment tooling, LLM use, and avoid duplicating the prediction model.
Backend	Database, event storage, APIs, authentication, order system	Implement fields/events and connect the prediction endpoint.
Virtual Assistant	Conversation flows, FAQs, onboarding and support dialogue	Collect required inputs, display predictions accurately, capture corrections.
UI/UX	Screens, flow, interaction design	Design prediction cards, confidence, explanations, reminder and feedback states.
Cybersecurity	Data protection, access, threat and risk controls	Review personal data, API security, logging, and retention.

Your specific ownership

You own the scientific definition of the prediction: what is predicted, which data is valid, how the cold start works, what baseline must be beaten, how performance is measured, and how the output is explained. You do not own survey charts or general business dashboards.

17. Open Product Decisions

The following decisions require PM and cross-functional agreement before implementation:

- Will users be allowed to log refills completed outside FlameIQ?
- Will one account support multiple cylinders during the MVP?
- How will partial refills be captured?
- What reminder threshold should trigger the first notification: predicted days, date range, or user preference?
- Can users choose conservative, balanced, or minimal reminder frequency?
- Will the product ask users to report the actual day gas finished, or rely only on the next refill date?
- Which onboarding questions are mandatory, and which can be skipped?
- What wording will prevent users from treating the estimate as a safety guarantee?
- Who is responsible for approving the synthetic data assumptions used in the hackathon demo?
- What is the fallback experience when the model service is unavailable or confidence is too low?

18. Final Recommendation

FlameIQ should begin with a transparent hybrid prediction system rather than an unnecessarily complex model. The first version should combine a cohort estimate for new users with personal refill history as it becomes available. The team should build and benchmark simple regression models only after the data schema and baseline are working.

For the hackathon, the strongest demonstration is not a high synthetic accuracy score. It is a complete, believable loop: the product collects the right data, produces an explainable estimate, communicates uncertainty, converts the estimate into a refill action, and learns from the eventual outcome.

Decision

Proceed with the refill prediction proof of concept using the staged cold-start strategy, a weighted hybrid baseline, and a small set of interpretable tabular models. Treat real-world accuracy as a pilot objective, not a hackathon claim.

Appendix A: Data Science Deliverables Checklist

- ☐ Prediction target and business interpretation approved
- ☐ Feature dictionary completed
- ☐ Backend events and required fields agreed
- ☐ Synthetic data assumptions documented
- ☐ Baseline implemented
- ☐ Candidate models evaluated on the same user-level split
- ☐ Selected model and rationale documented
- ☐ Prediction range and confidence logic implemented
- ☐ Explainability message implemented
- ☐ API request and response tested
- ☐ Prediction logs captured
- ☐ Limitations and synthetic-data disclosure included in pitch